

NED UNIVERSITY OF ENGINEERING & TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE & IT

TS (COMPUTER SCIENCE)

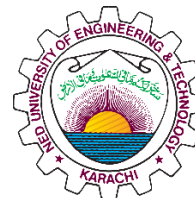
CT-367 THEORY OF PROGRAMMING LANGUAGES

REPORT

**A FORMAL LEXICAL ANALYZER FOR THE .ENV FILE
CONFIGURATION**

SYED MUHAMMED AHMED
ABDUR REHMAN SIDDIQUI
QAZI ASIM KAMAL
MUHAMMED AFFAN

CT-23049
CT-23037
CT-23044
CT-23040



**DEPARTMENT OF COMPUTER SCIENCE & INFORMATION TECHNOLOGY
BACHELORS OF SCIENCE IN COMPUTER SCIENCE**

Complex Computing Problem Assessment Rubrics

Course Code: CT-367		Course Title: Theory of Programming Language	
Criteria and Scales			
Excellent (3)	Good (2)	Average (1)	Poor (0)
<u>Criterion 1:</u> Understanding the Problem: How well the problem statement is understood by the student			
Understands the problem clearly and clearly identifies the underlying issues and functionalities.	Adequately understands the problem and identifies the underlying issues and functionalities.	Inadequately defines the problem and identifies the underlying issues and functionalities.	Fails to define the problem adequately and does not identify the underlying issues and functionalities.
<u>Criterion 2:</u> Research: The amount of research that is used in solving the problem			
Contains all the information needed for solving the problem	Good research, leading to a successful solution	Mediocre research which may or may not lead to an adequate solution	No apparent research
<u>Criterion 3:</u> Code: How complete the code is along with the assumptions and selected functionalities			
Complete Code according to the according to the selected functionalities of the given case with clear assumptions	Incomplete Code according to the selected functionalities of the given case with clear assumptions	Incomplete Code according to the selected functionalities of the given case with unclear assumptions	Wrong code and naming conventions
<u>Criterion 4:</u> Report: How thorough and well organized is the solution			
All the necessary information clearly organized for easy use in solving the problem	Good information organized well that could lead to a good solution	Mediocre information which may or may not lead to a solution	No report provided

Total Marks: _____

Teacher's Signature: _____

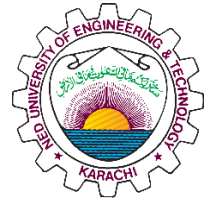
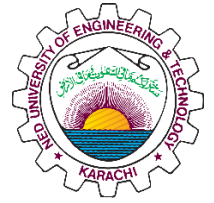


TABLE OF CONTENTS

No.	Section	Page
01	Problem Description	03
02	Example Problem String (Node.js, Python, Bash, Docker)	04
03	Scope of Project	05
04	Code Snippets	06
05	Assumptions	08
06	Limitations	09
07	Future Scope	10
08	References	12



1. PROBLEM DESCRIPTION

The .env file format is a widely adopted convention for storing environment variables, such as database credentials, API keys, and configuration flags across virtually every modern software project. Platforms including Node.js, Python, Docker, Ruby, and Bash all rely on .env files, making it one of the most universally used configuration mechanisms in software development.

The core problem is the absence of a canonical, language-neutral token grammar for the .env format. Without one, every parser author must make independent assumptions about edge cases, quoting styles, escape sequences, variable expansion, whitespace handling and those assumptions diverge. This project addresses that gap by constructing a formal lexical specification using Flex (Fast Lexical Analyzer).

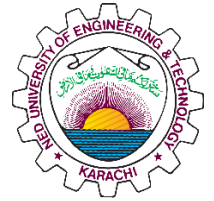
2. REAL WORLD EXAMPLE

Consider the developer who stores the database password containing the dollar sign.

Tokenization comparison across all platforms:

Platform	Reads DB_PASSWORD as	Behavior
Node.js dotenv	p@\$word	\$ treated as literal
Docker Compose	p@word	\$\$ collapsed to \$, then shell-expanded
Python dotenv	p@\$word	\$ treated as literal
Bash (source .env)	p@word	Shell variable expansion consumes \$\$

The consequence: The application works perfectly in development on Node.js. It is deployed using Docker Compose. The database connection fails in production because Docker silently reads a different password from the exact same file. The developer spends hours debugging not because of a code bug, but because two tools disagree on what the same token means.



3. SCOPE OF PROJECT

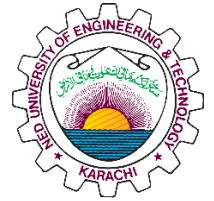
This project delivers a formal lexical analyzer for the .env configuration language, built using Flex (Fast Lexical Analyzer). The scope is defined as follows:

3.1 In Scope

- Designing a formal token grammar covering all .env token types using regular expressions
- Implementing the grammar as a working Flex (.l) source file
- Handling all major quoting styles: unquoted values, single-quoted values, and double-quoted values
- Recognizing escape sequences within double-quoted strings (`\n`, `\t`, `\"`, `\\`, `\$`)
- Handling the export keyword as an optional prefix for key-value pairs
- Tokenizing inline comments and standalone comment lines
- Detecting and classifying variable expansion syntax (`${VAR}`)
- Producing structured token output: token type, line number, and lexeme
- Providing a test suite of .env files covering standard cases and edge cases
- Documenting all design decisions, token definitions, assumptions, and limitations

3.2 Out Scope

- Parsing (syntax analysis), this project covers lexical analysis only
- Semantic interpretation or runtime variable substitution
- Integration with any existing parser library (dotenv, python-dotenv, etc.)
- Support for non-standard extensions such as multiline values with backslash continuation
- A GUI or web interface for the lexer
- Cross-compilation or packaging for production deployment



4. CODE SNIPPETS (LEXICAL RULES AND TOKEN GRAMMER)

4.1 TOKEN DEFINITIONS (Flex .l File Definitions Section)

```
%x VALUE_STATE

KEY      [A-Z_][A-Z0-9_]*
WS       [ \t]+
NEWLINE  \n
COMMENT  #[^\n]*
EXPORT_KW export
ESC_SEQ  \\[ntrfb\\"]
```

4.2 Lexer Rules (Flex .l File Rules Section)

```
<INITIAL>{WS}      { /* skip leading whitespace */ }

<INITIAL>{COMMENT}  { print_token("COMMENT", yytext, NULL); }

<INITIAL>{EXPORT_KW}/{WS} { print_token("EXPORT_KW", yytext, NULL); }

<INITIAL>{KEY}      { print_token("KEY", yytext, NULL); }

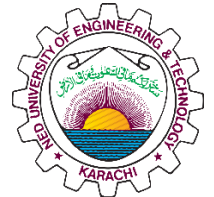
<INITIAL>"="        {
    print_token("EQUALS", yytext, NULL);
    BEGIN(VALUE_STATE);
}

<INITIAL>{NEWLINE}  {
    printf(" [line %d] NEWLINE\n", line_num);
    line_num++;
}

<INITIAL>[a-z_][a-zA-Z0-9_]* {
    print_token("INVALID_KEY", yytext, "RISK: lowercase key, not portable");
    invalid_count++;
}

<INITIAL>[A-Z_][A-Z0-9_]*[a-z][^\n]* {
    print_token("INVALID_KEY", yytext, "RISK: mixed-case key, not portable");
    invalid_count++;
}

<INITIAL>.,         {
    print_token("INVALID", yytext,
        "RISK: unparseable token");
    invalid_count++;
}
```



```
1921-2021
<VALUE_STATE>(WS)      { /* skip whitespace after = */ }

<VALUE_STATE>'[^\\n]*'  {
    print_token("SINGLE_QUOTED", yytext, NULL);
    BEGIN(INITIAL);
}

<VALUE_STATE>'[^\\n']*\\n {
    print_token("INVALID", yytext,
        "RISK: unclosed single-quoted string");
    invalid_count++;
    line_num++;
    BEGIN(INITIAL);
}

<VALUE_STATE>\"{([ESC_SEQ]|[^\\n])*\n\" {
    print_token("DOUBLE_QUOTED", yytext, NULL);
    BEGIN(INITIAL);
}

<VALUE_STATE>\"{([ESC_SEQ]|[^\\n])*\n\" {
    int i;
    print_token("MULTILINE_VAL", yytext, NULL);
    for (i = 0; yytext[i]; i++)
        if (yytext[i] == '\\n') line_num++;
    BEGIN(INITIAL);
}
```

```
<VALUE_STATE>\"{[^\\n]*\\n {
    print_token("INVALID", yytext,
        "RISK: unclosed double-quoted string");
    invalid_count++;
    line_num++;
    BEGIN(INITIAL);
}

<VALUE_STATE>\\$[A-Z_][A-Z0-9_]* {
    print_token("VAR_EXPANSION", yytext,
        "RISK: platform-dependent -- expanded by Docker/Bash, literal in Node/Python");
    risk_count++;
}

<VALUE_STATE>\\$[A-Z_][A-Z0-9_]* {
    print_token("VAR_SHORT", yytext,
        "RISK: platform-dependent -- boundary ambiguous in shell");
    risk_count++;
}

<VALUE_STATE>\\$ {
    print_token("INVALID", yytext,
        "RISK: bare $ -- undefined behavior across platforms");
    invalid_count++;
}

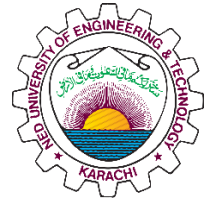
<VALUE_STATE>[^\\n#\"'\\$\\t ]+ {
    print_token("UNQUOTED_VAL", yytext, NULL);
}
```

```
<VALUE_STATE>[ \\t]*(COMMENT) {
    const char *c = yytext;
    while (*c == ' ' || *c == '\\t') c++;
    print_token("INLINE_CMT", c, NULL);
    BEGIN(INITIAL);
}

<VALUE_STATE>{NEWLINE} {
    printf(" [line %d] NEWLINE\\n", line_num);
    line_num++;
    BEGIN(INITIAL);
}

<VALUE_STATE>\\. {
    print_token("INVALID", yytext,
        "RISK: unparseable token in value");
    invalid_count++;
}

%%
```



4.3 Helper Functions

```
void print_token(const char *type, const char *value, const char *risk) {
    token_count++;
    if (risk) {
        printf(" [line %2d] %-14s | %-30s | !! %s\n",
            line_num, type, value, risk);
    } else {
        printf(" [line %2d] %-14s | %s\n",
            line_num, type, value);
    }
}

void print_sep() {
    int i;
    printf(" ");
    for (i = 0; i < 88; i++) printf("-");
    printf("\n");
}
```

4.4 Main Function

```
int yywrap() { return 1; }

int main(int argc, char *argv[]) {
    if (argc > 1) {
        FILE *f = fopen(argv[1], "r");
        if (!f) {
            fprintf(stderr, "Error: cannot open file '%s'\n", argv[1]);
            return 1;
        }
        yyin = f;
    }

    printf("\n");
    printf(" +-----+\n");
    printf(" | .env Formal Lexical Analyzer CT-367 | \n");
    printf(" +-----+\n");
    printf("\n");
    print_sep();
    printf(" [line ##] TOKEN TYPE      | VALUE                                | RISK\n");
    print_sep();

    yylex();
}
```

```

print_sep();
printf("\n");
printf(" SUMMARY\n");
printf(" ----- \n");
printf(" Total lines      : %d\n", line_num - 1);
printf(" Total tokens     : %d\n", token_count);
printf(" Risk sites       : %d (lines where platforms disagree)\n", risk_count);
printf(" Invalid tokens    : %d (lines no platform can parse cleanly)\n", invalid_count);
printf("\n");

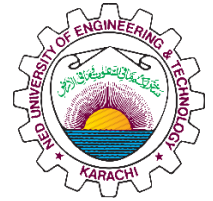
if (risk_count == 0 && invalid_count == 0) {
    printf(" CLEAN: This .env file is safe across Node.js, Python, Docker, and Bash.\n");
} else {
    printf(" WARNING: Review flagged tokens before deploying to multi-platform environment.\n");
}
printf("\n");

return 0;
}

```

4.5 Token Type Summary

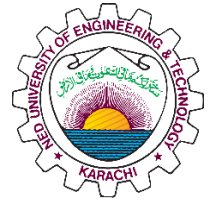
Token	Example	Regular Expression
COMMENT	# This is a comment	#[^\n]*
EXPORT_KW	export	export
KEY	DATABASE_URL	[A-Z_][A-Z0-9_]*
EQUALS	=	=
UNQUOTED_VALUE	hello123	[^\n#"]+
SINGLE_QUOTED_VALUE	'hello world'	'[^']*'
DOUBLE_QUOTED_VALUE	"hello\nworld"	"(\\. [^\\"\\"])*"
VAR_EXPANSION	\${PORT}	\\${[A-Z_]+\}
NEWLINE	(line ending)	\n
WHITESPACE	(spaces/tabs)	[\t]+
INVALID	(malformed input)	catch-all



4.6 Sample Tokenizer Output

```
+-----+
| .env Formal Lexical Analyzer  CT-367  |
+-----+

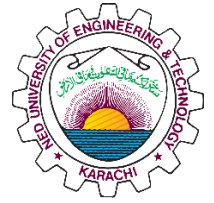
-----
[Line #]  TOKEN TYPE  | VALUE                                     | RISK
-----
[Line 1]  COMMENT     | # =====
[Line 1]  NEWLINE
[Line 2]  COMMENT     | # sample.env Test input for the CT-367 .env Lexer
[Line 2]  NEWLINE
[Line 3]  COMMENT     | # This file deliberately contains all token types including
[Line 3]  NEWLINE
[Line 4]  COMMENT     | # cross-platform risk sites for demonstration purposes.
[Line 4]  NEWLINE
[Line 5]  COMMENT     | # =====
[Line 5]  NEWLINE
[Line 6]  NEWLINE
[Line 7]  COMMENT     | # Section 1: Clean, safe assignments
[Line 7]  NEWLINE
[Line 8]  KEY         | DB_HOST
[Line 8]  EQUALS      | =
[Line 8]  UNQUOTED_VAL | localhost
[Line 8]  NEWLINE
[Line 9]  KEY         | DB_PORT
[Line 9]  EQUALS      | =
[Line 9]  UNQUOTED_VAL | 5432
[Line 9]  NEWLINE
[Line 10] KEY         | APP_NAME
[Line 10] EQUALS      | =
[Line 10] SINGLE_QUOTED | 'My Application'
[Line 10] NEWLINE
[Line 11] KEY         | DESCRIPTION
[Line 11] EQUALS      | =
[Line 11] DOUBLE_QUOTED | "A production web server"
[Line 11] NEWLINE
[Line 12] NEWLINE
```



5. ASSUMPTIONS

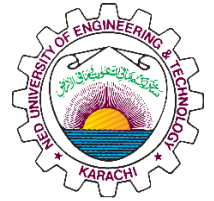
The following assumptions underpin the design and implementation of the lexer:

- Keys are assumed to follow the POSIX naming convention, uppercase letters, digits, and underscores only, beginning with a letter or underscore. Lowercase keys, while accepted by some tools, are treated as INVALID tokens to enforce a canonical standard.
- Whitespace (spaces and tabs) surrounding the = sign is non-semantic and is suppressed not emitted as a token. This matches the behavior of Node.js dotenv and python-dotenv.
- The \$\$ sequence is treated as a risk site and flagged with a platform-dependency warning, since its behavior is undefined across Node.js, Docker, and Bash.
- Escape sequences (\n, \t, \", \\, \\$) are recognized only within double-quoted values. They have no special meaning in single-quoted values or unquoted values.
- Single-quoted values are treated as fully literal, no escape sequences, no variable expansion, no special characters.
- Variable expansion syntax (\${VAR}) is recognized and tokenized as VAR_EXPANSION. The lexer does not perform the expansion itself that is a semantic-layer responsibility.
- Each logical line of the .env file corresponds to at most one key-value pair. Multiline values (continuation via backslash) are not supported in this version.
- The export keyword, when present, is an optional syntactic prefix and does not alter the semantics of the key-value pair. It is emitted as EXPORT_KW.
- Blank lines and lines containing only whitespace are consumed and produce only NEWLINE tokens, with no other output.
- The input encoding is assumed to be UTF-8 / ASCII. Non-ASCII characters in keys or values are classified as INVALID.



6. LIMITATIONS

- The lexer performs lexical analysis only. It does not perform parsing, semantic validation, or runtime variable resolution. A full .env interpreter would require an additional parsing and evaluation layer.
- Multiline values, achieved in some implementations by ending a line with a backslash continuation character are not supported. Such input will be tokenized incorrectly across the line boundary.
- Variable expansion ($\${VAR}$) is recognized as a token but not evaluated. The actual value substitution is outside the scope of a lexical analyzer.
- Lowercase keys are marked INVALID. While some platforms (e.g., Docker Compose) permit lowercase variable names, this lexer enforces the uppercase-only convention for canonical compliance.
- There is no support for nested or recursive variable expansions such as $\${\${BASE}}$.
- The INVALID catch-all rule provides a token type and line number but does not attempt error recovery. Lexing continues on the next character, which may produce cascading INVALID tokens.
- Unicode character classes are not supported. The KEY rule uses $[A-Z_][A-Z0-9_]*$, which does not cover extended Unicode identifiers.



7. FUTURE SCOPE

The following directions represent potential extensions of this work:

7.1 Parser Integration

The token stream produced by this lexer is the foundation for a full .env parser. A future extension would implement a context-free grammar (CFG) in Bison that consumes this token stream and produces a structured AST or symbol table of key-value mappings.

7.2 Semantic Layer & Variable Resolution

Once a parser exists, a semantic evaluation pass could implement variable expansion, resolving $\{VAR\}$ tokens by looking up previously defined keys in the same file. This would bring the analyzer to feature parity with tools like python-dotenv.

7.3 Cross-Platform Compatibility Mode

The lexer could be extended with a platform flag (`--mode=docker`, `--mode=bash`, `--mode=node`) that adjusts tokenization rules to emulate each platform's behavior. This would make the tool useful as a debugging aid, developers could run their .env file through each mode and see exactly how values would differ.

7.4 Lowercase Key Support

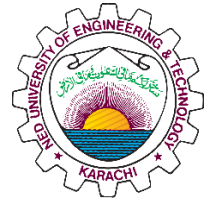
Relaxing the KEY rule to `[A-Za-z_][A-Za-z0-9_]*` would bring the lexer into compliance with platforms that accept lowercase variable names. This is a small change to the grammar but has broad compatibility implications.

7.5 Inline Comment Handling

Supporting end-of-line comments (`VALUE=foo # comment`) requires the lexer to be aware of whether it is inside a value context. This can be achieved using Flex start conditions to switch between a `KEY_CONTEXT` and `VALUE_CONTEXT` state, enabling comment stripping at the end of unquoted values.

7.6 Multiline Value Support

Backslash-continuation multiline values are supported by some parsers. A future version could use Flex start conditions to enter a `MULTILINE` state when a value line ends with `\`, consuming subsequent lines until a non-escaped newline is encountered.



7.7 Formal Specification Document

The ultimate deliverable of this research direction is a language-neutral formal specification document similar to an IETF RFC, that could be adopted by the maintainers of Node.js dotenv, python-dotenv, Docker Compose, and other tools to align their parsing behavior.

8. REFERENCES

1. Node.js Official Documentation (Environment Variables)
https://nodejs.org/api/environment_variables.html
2. Flex Manual (The Fast Lexical Analyzer)
<https://westes.github.io/flex/manual/>
3. motdotla/dotenv (Node.js dotenv Source)
<https://github.com/motdotla/dotenv>
4. python-dotenv Documentation
<https://saurabh-kumar.com/python-dotenv/>
5. Docker Compose (Environment Variables Reference)
<https://docs.docker.com/compose/environment-variables/>
6. Aho, Lam, Sethi, Ullman, Compilers: Principles, Techniques, and Tools (Dragon Book), 2nd Edition. Pearson, 2006.